

CivicPulse AI

Issue Reporting Module

Research & Technical Design

CivicPulse AI is an AI-powered civic issue reporting and management platform. Citizens report problems — potholes, garbage, broken streetlights, water leakage, damaged roads — and the system routes each report through automated analysis to the responsible civic authority.

This module — Issue Reporting — is the entry point of the entire platform. It turns a citizen's real-world observation into a structured, geolocated, evidence-backed digital record that every downstream module (AI analysis, duplicate detection, priority, authority assignment, dashboard) depends on.

In one sentence: a citizen describes a problem, attaches a photo and a location, submits it, receives a trackable Issue ID, and the rest of CivicPulse takes over from there.

Prepared for	CivicPulse AI Development Team (5 members)
Module owner	Urvesh — Issue Reporting
Stack	React · Node.js/Express · PostgreSQL+PostGIS · Python AI · REST · JWT
Status	Pre-implementation research & design

01 · PROBLEM & OBJECTIVE

Why Issue Reporting Needs a Redesign

Problems with traditional civic reporting		How CivicPulse addresses each one
Problem	Real-world effect	CivicPulse mechanism
Difficult reporting process	Citizens give up before filing a complaint	Guided, mobile-first reporting flow
Inaccurate location	Field crews sent to the wrong spot	Device GPS + PostGIS-backed map confirmation
Lack of evidence	Authorities cannot verify severity	Mandatory photo evidence with upload validation
Duplicate complaints	Same pothole logged 50 times, wastes effort	Spatial duplicate detection (PostGIS proximity search)
Poor categorization	Issue routed to the wrong department	AI-assisted category suggestion, human-confirmed
No transparent tracking	Citizens lose trust in the system	Status lifecycle with Issue ID + push/track updates
Poor routing / prioritization	Urgent issues wait behind trivial ones	AI-driven priority scoring & authority assignment

Module objective

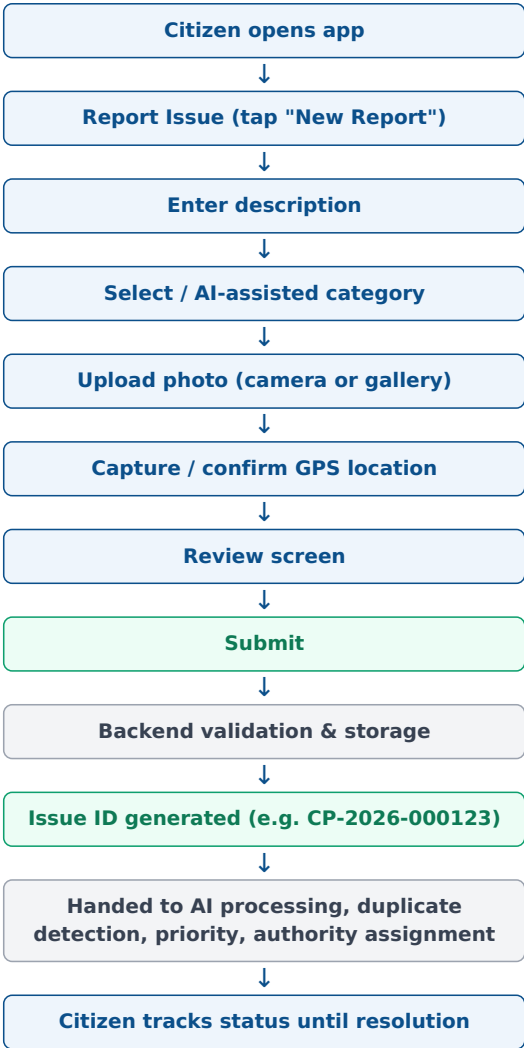
"The Issue Reporting module converts a citizen's real-world civic problem into a structured, geolocated, evidence-backed digital issue that can be processed by the rest of the CivicPulse system."

What this module is *not*

It is not the AI classifier, not the duplicate-detection engine, and not the dashboard. Issue Reporting's job ends the moment a clean, valid, structured issue record exists in the database with status **SUBMITTED**. Everything after that is another module's responsibility — but this module must produce output that those modules can trust.

Grounding note: this problem framing is informed by patterns observed in established civic-reporting platforms (Open311/FixMyStreet in the UK/US, and India's Swachhata-MoHUA app) — see Sources on the final page.

Complete User & System Workflow



Step notes

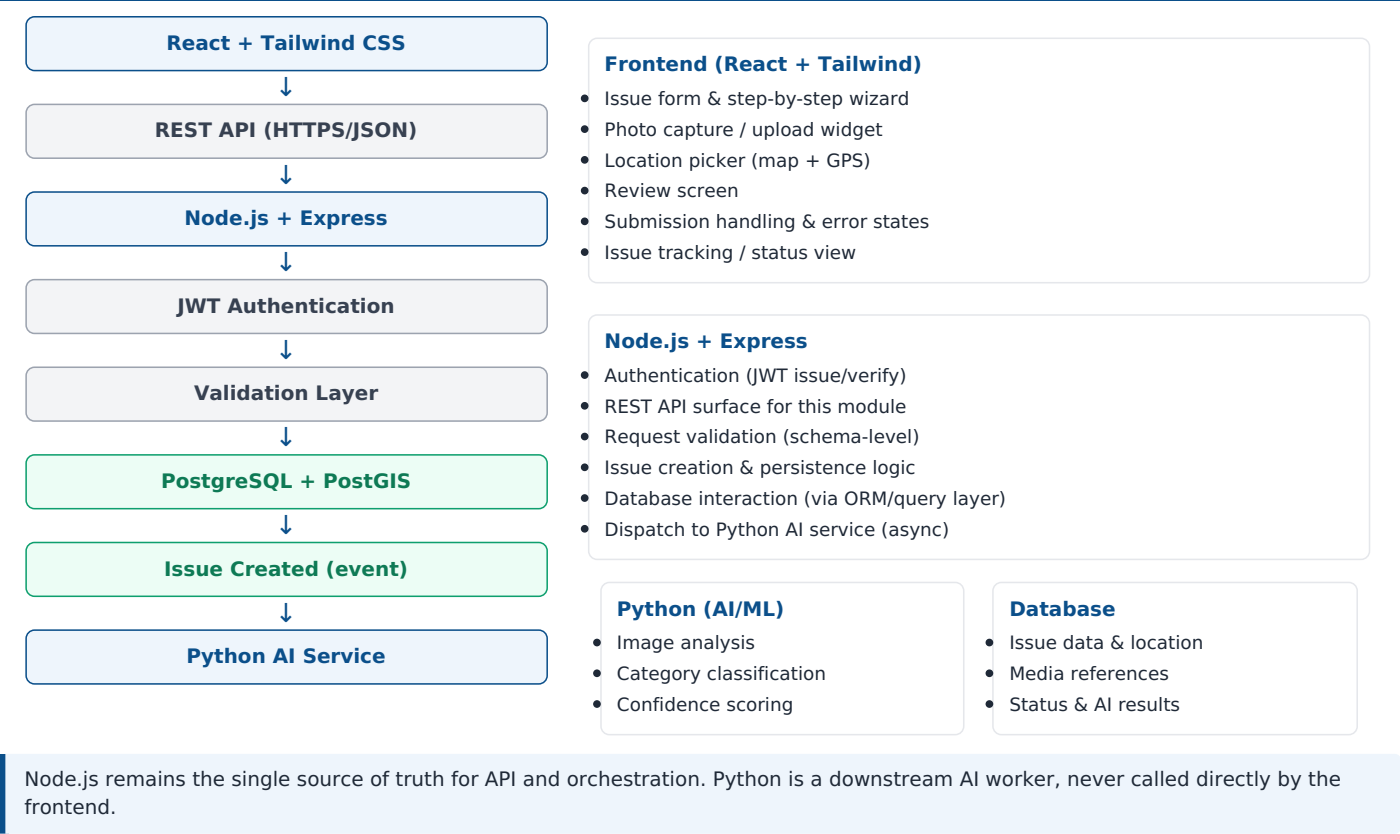
- **Description:** free text, minimum length enforced client-side.
- **Category:** citizen picks from a fixed list; AI later re-classifies and can flag mismatches for review — the citizen's pick is never overwritten silently.
- **Photo:** required for MVP (configurable per category later); validated for type/size before upload.
- **Location:** auto-captured via device GPS, editable via a pin-drop map (PostGIS-backed) if GPS is inaccurate.
- **Review:** last chance to edit before the record is written — prevents wasted backend writes.
- **Issue ID:** the single identifier every other module references.

Sample user journey

Citizen finds a pothole → opens CivicPulse → uploads a photo → GPS auto-fills location → reviews and submits → receives CP-2026-000123 → gets notified as the issue moves from *Submitted* to *Assigned* to *Resolved*.

03 · SYSTEM ARCHITECTURE

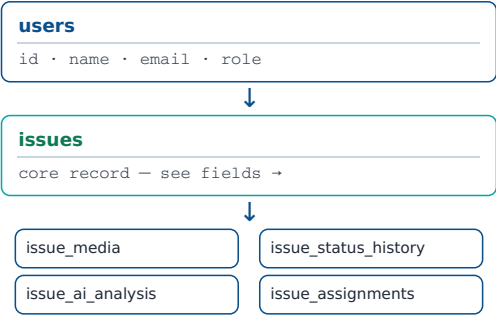
Frontend + Backend Architecture



04 • DATA MODEL

Database & PostGIS Design

Entity relationships



Core issues fields

issue_id
user_id
description
category
subcategory
latitude
longitude
location <i>(PostGIS geometry)</i>
status
created_at
updated_at

Downstream fields

(optional/nullable)

priority
ai_category
ai_confidence
duplicate_of
authority_id
ai_status

These are written by other modules, not by Issue Reporting — but the columns live on the row this module creates.

Why PostGIS

- Stores geographic location as real spatial data, not just two floats
- Finds nearby issues with radius/proximity queries
- Calculates distance between reports accurately
- Supports duplicate detection (module 2)
- Enables map-based analytics (dashboard module)

Example

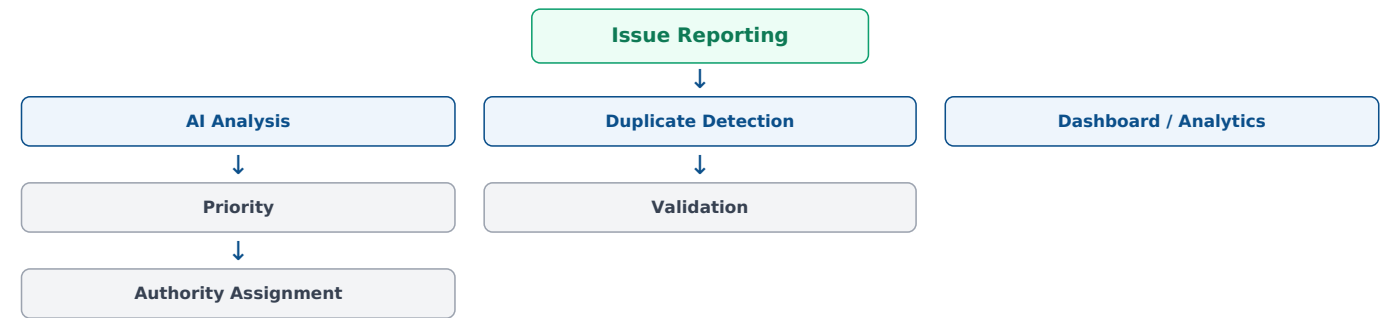
New pothole reported → coordinates stored as a PostGIS point → a `ST_DWithin` query searches existing issues within ~25m → candidates are passed to the Duplicate Detection module for a final decision.

05 · API CONTRACT & INTEGRATION

Proposed REST APIs TO BE FINALIZED WITH TEAM

Endpoint	Purpose	Main input	Main output
POST /api/issues	Create a new issue	description, category, photo(s), lat/lng	issue_id, status = SUBMITTED
GET /api/issues/:id	Fetch one issue	issue_id (path)	full issue record
GET /api/issues/my	List current user's issues	JWT (implicit), pagination params	array of issue summaries
GET /api/issues/:id/status	Poll/track status	issue_id (path)	status, status_history
POST /api/issues/:id/media	Attach additional media	issue_id, file	media reference, updated media list

Module integration



Target module	Data Issue Reporting sends
AI Analysis	issue_id, photo(s), description, citizen-selected category
Duplicate Detection	issue_id, location (PostGIS point), category, created_at
Priority Management	issue_id, ai_category, ai_confidence (once available)
Authority Assignment	issue_id, category, location (to resolve jurisdiction)
Dashboard / Analytics	issue_id, category, status, location, timestamps

06 · LIFECYCLE & FAILURE HANDLING

Issue Status Lifecycle



State	Meaning
SUBMITTED	Saved successfully; awaiting review
UNDER_REVIEW	AI / moderation checks in progress
VERIFIED	Confirmed genuine, not spam/duplicate
ASSIGNED	Routed to a responsible authority
IN_PROGRESS	Authority is actively working on it
RESOLVED	Fix completed, pending citizen confirmation
CLOSED	Final state, no further action
DUPLICATE	Merged into an existing issue
REJECTED	Invalid, spam, or out of scope

Failure cases to design for

- GPS permission denied
- Incorrect / low-accuracy GPS
- No photo attached
- Invalid or oversized file
- Empty description
- Duplicate submission (double-tap)
- Spam submissions
- Unauthorized request (bad/expired JWT)
- Network failure mid-submit
- Database failure
- AI service unavailable

Architectural rule: AI failure must NOT cause loss of the citizen's report.
Example → Issue = SAVED, AI Status = PENDING. The AI service can process it later; the citizen's submission is never blocked or discarded because a downstream service is down.

07 · SECURITY & DELIVERY PLAN

Security Checklist

- ☐ JWT authentication on every write endpoint
- ☐ Authorization (users edit only their own issues)
- ☐ Input validation (schema-level, server-side)
- ☐ File type & size validation on uploads
- ☐ Rate limiting on submission endpoints
- ☐ Secure, access-controlled media storage
- ☐ SQL injection protection (parameterized queries)
- ☐ Citizen location/PII privacy handling

Implementation Roadmap

Phase	Deliverable
1 · Research	This document + finalized requirements
2 · UI / Wireframe	Approved wireframes for the reporting flow
3 · Database + PostGIS	Finalized schema, migrations applied
4 · REST APIs	API contract signed off by all 5 members
5 · React implementation	Working reporting UI against mock API
6 · Backend implementation	Live endpoints backed by PostgreSQL
7 · Media + Location	Upload pipeline + map/GPS picker working
8 · AI integration	Async handoff to Python service functioning
9 · Team integration	Verified handoff to all four other modules
10 · Testing + Docker	Containerized, tested, ready to demo

My Responsibility vs. the Team's

Issue Reporting (mine)

- Reporting UI & step-by-step form
- Description & category input
- Media upload
- Location selection (GPS + map)
- Review screen
- Submission handling
- Issue-creation API
- Issue database structure
- Issue ID generation
- Initial status management
- Issue retrieval / tracking endpoints
- Integration contracts with other modules

AI

- Classification
- Image analysis
- AI confidence scoring

Duplicate Detection

- Duplicate identification logic

Priority / Authority

- Priority calculation
- Authority assignment

Dashboard

- Maps
- Analytics
- Visualization

Exact responsibilities may be adjusted during team integration as interfaces are finalized together.

Key Takeaways

1. Issue Reporting is more than a form — it is the data contract for the whole platform.

2. Every issue must become clean, structured data.

3. Location is a core component, not an afterthought.

4. Evidence (photos) improves verification and trust.

5. PostGIS enables all spatial operations downstream.
6. Node.js is the primary backend and orchestrator.

7. Python handles AI/ML only — never called directly by the client.

8. AI failure must never cause loss of a citizen's report.

9. The Issue ID connects all modules to a single source of truth.

10. API contracts, agreed early, are what let 5 people build in parallel.

Development Checklist

- ☐ Requirements finalized
- ☐ User flow finalized
- ☐ UI wireframe completed
- ☐ Database schema finalized
- ☐ PostGIS design finalized
- ☐ API contract finalized
- ☐ Frontend completed
- ☐ Backend completed
- ☐ Media / location completed
- ☐ AI integration completed
- ☐ Team integration completed
- ☐ Testing completed
- ☐ Docker / GitHub setup completed

Sources & reference systems

System	Type	What we observed
Open311 (GeoReport v2)	Existing system	An open, widely-adopted API standard for civic issue tracking, used by cities like Chicago, Boston and Washington D.C.; our REST design borrows its spirit of a simple, jurisdiction-agnostic issue API, without adopting the spec verbatim.
FixMyStreet (UK)	Existing system	A citizen-facing reporting platform that connects to council backends via Open311, confirming that a clean create/track API pair is enough to support a full reporting product.
Swachhata-MoHUA (India)	Existing system	A government-backed app for sanitation issue reporting with photo + GPS capture and a unique tracking ID per complaint — validates our photo-plus-location-plus-ID approach for the Indian civic context.
CivicPulse AI design (this doc)	Proposed	AI-assisted categorization, PostGIS-based duplicate detection, and automated priority/authority routing — not yet implemented.
Community voting, badges, offline queueing	Future / optional	Observed in some existing apps; not in MVP scope for this module.

This document reflects proposed design only. Nothing described as "proposed" above is implemented yet; all API shapes and schemas are to be finalized with the team before Phase 4.